🔍 : GitHub Explorer with Promises

Scenario: You're building a lightweight GitHub Explorer that fetches user info, their repositories, and the primary language for each repo—using only Promises and the Fetch API.

Your Tasks

1. **Fetch User Profile**
   - Write a function `fetchGitHubUser(username)` that returns a Promise resolving to the parsed JSON from

     ```
     https://api.github.com/users/{username}
     ```

   - If `response.ok` is false, reject with `"User not found"`.
2. **Fetch Repositories**
   - Write `fetchRepos(user)` that takes the user object and returns a Promise resolving to the array from

     ```
     https://api.github.com/users/{username}/repos
     ```

   - Again, reject if the response isn't OK.
3. **Fetch Repository Languages**
   - For each repo in the array, fetch its languages via

     ```
     https://api.github.com/repos/{username}/{repoName}/languages
     ```

   - Write `fetchRepoLanguages(repo)` returning a Promise for that JSON.
4. **Aggregate Results**
   - Chain the above calls to:
     1. Fetch the user "octocat".
     2. Fetch their repos.
     3. For the **first five** repos, fetch languages in parallel using `Promise.all`.
     4. Log an array of objects:

```
[
  { name: 'repo1', languages: { JavaScript: 12345, HTML: 6789, … }
},
  … up to five entries …
]
```

5. **Error Handling**
   - If **any** fetch fails (user not found, repos error, or languages error), catch it at the appropriate level and log a helpful message (e.g. `"Failed to fetch languages for repo1"`), but still attempt to load the others.
6. **Finally Cleanup**
   - In a `.finally()`, log `"GitHub Explorer operation complete"` regardless of success or failure.

What You'll Practice:

- Using `fetch` with Promises (`.then`, `.catch`, `.finally`)
- Parallel requests with `Promise.all`
- Graceful degradation: handling individual failures without aborting the entire flow
- Interacting with a real public API and parsing JSON responses